

Personal Finance & Expense Intelligence Dashboard

A full-stack Python engineering project

Flask · Streamlit · SQLite · Pandas · NumPy · Matplotlib · Seaborn · Scikit-learn

Built to practice: OOP · Generators · Decorators · File Handling · Logging · Multithreading · Exception Handling · Memory Management

Engineering Major Project Blueprint

Module 1
Database

Module 2
Core Engine

Module 3
Analytics

Module 4
Web UI

Module 5
ML Layer

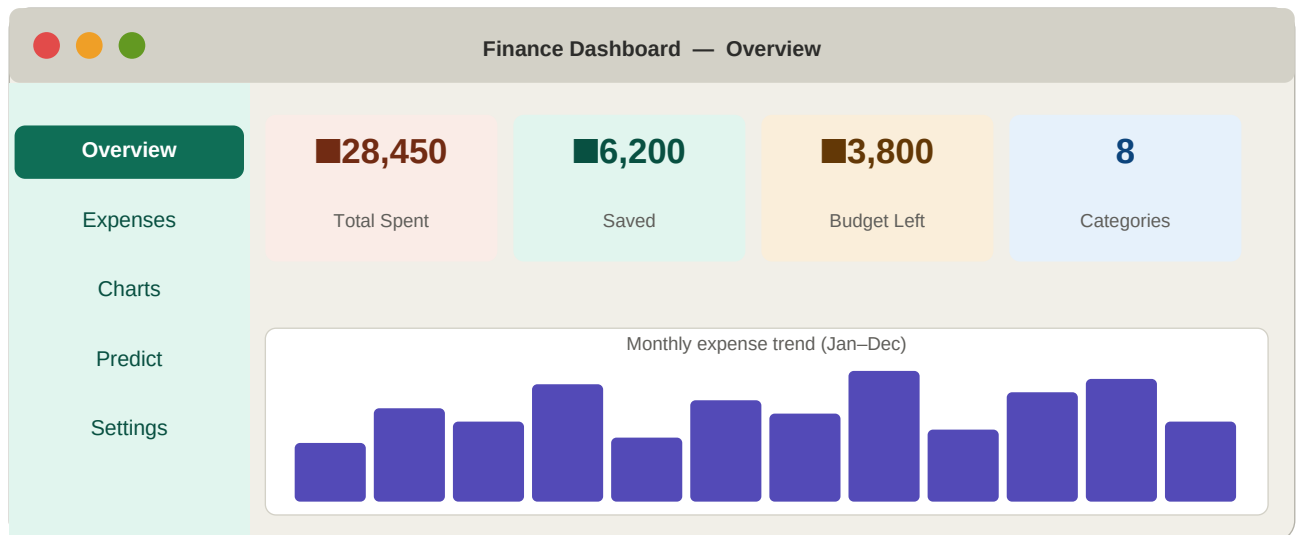
What is this project?

The Personal Finance & Expense Intelligence Dashboard is a full-stack Python application that lets users log daily expenses, visualize spending patterns, set budgets, and get predictions on future spending using machine learning. It is designed as a major engineering project that demonstrates mastery of core Python concepts while delivering a real, usable product.

A user opens the Streamlit dashboard, adds today's expenses (food ■180, transport ■60), and instantly sees a chart of this month vs last month. The system flags that food spending is 40% above average and predicts they will overshoot their budget by ■2,000 before month-end. All of this runs locally — no cloud, no subscription, no data leaving the device.

What the finished product looks like

Below is a preview of the dashboard UI — four metric cards at the top, a sidebar for navigation, and a monthly bar chart as the main visual. This is what you are building.

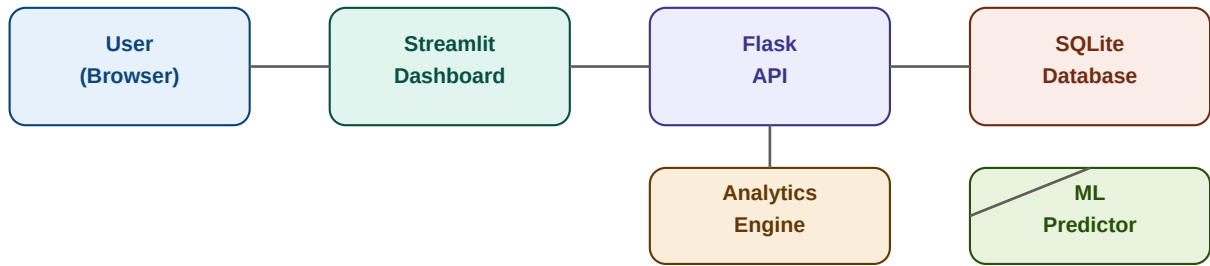


The dashboard will have 4 pages: Overview, Expenses, Charts, and Predictions.

How the layers connect

The architecture below shows how every part of the system talks to each other. The user interacts with Streamlit, which calls Flask APIs, which read/write SQLite, while the Analytics Engine and ML Predictor work in the background.

System architecture — how the layers connect



Project modules — build order

The project is split into 5 modules. Build them in order — each module depends on the previous. Use this section as your checklist as you progress.

Module 1 — Data Layer

The foundation of the entire project. Defines the SQLite database schema, OOP models (User, Expense, Category, Budget), and the DatabaseManager class that handles all CRUD operations with logging and exception handling throughout.

sqlite3 • OOP • file handling • logging • exception handling

Module 2 — Core Engine

The brain of the app. Generators and iterators for memory-efficient data streaming. Custom decorators for input validation and auth checking. Custom exception classes. File handler for CSV import/export. Memory management best practices throughout.

generators • iterators • decorators • memory management • exception handling

Module 3 — Analytics

Turns raw data into insights. Pandas for data wrangling and monthly aggregations. NumPy for statistical calculations (averages, standard deviations, trend detection). Matplotlib and Seaborn for generating charts: bar charts, pie charts, heatmaps, trend lines.

pandas • numpy • matplotlib • seaborn

Module 4 — Web Interface

The face of the product. Streamlit dashboard with 4 pages. Flask REST API with endpoints for add, edit, delete, and summary. Multithreading for background report generation so the UI never freezes. All pages connect to the database through the Flask API.

streamlit • flask • multithreading • REST API

Module 5 — ML Layer

The impressive extra layer. A Linear Regression model trained on the user's own expense history to predict next month's spend by category. An anomaly detector that flags unusual expenses vs the personal historical average. Adds 'ML-powered predictions' to your resume.

scikit-learn • linear regression • anomaly detection • prediction

Folder structure

Create this exact folder structure before writing any code. Every file has one clear responsibility.

finance_dashboard/	root project folder
database/	Module 1 — all database files
db_manager.py	DatabaseManager class, all CRUD operations
models.py	OOP classes: User, Expense, Category, Budget
schema.sql	CREATE TABLE statements
seed_data.py	populate DB with sample test data
core/	Module 2 — logic engine
expense_engine.py	generators & iterators for large data
decorators.py	@validate_input, @log_activity, @require_auth
file_handler.py	CSV import/export, file read/write
exceptions.py	custom exceptions: InvalidExpenseError etc.
analytics/	Module 3 — data analysis
reports.py	pandas: monthly summaries, category totals
charts.py	matplotlib + seaborn chart generators
stats.py	numpy: averages, trends, anomaly detection
api/	Module 4 — Flask backend
app.py	Flask app entry point, blueprint registration
routes.py	REST endpoints: /add, /edit, /delete
background.py	multithreading: async report generation
dashboard/	Module 4 — Streamlit frontend
streamlit_app.py	main Streamlit UI file, page routing
pages/1_overview.py	summary cards: total spend, savings rate
pages/2_expenses.py	add/edit/view expense table
pages/3_charts.py	visual analytics page
pages/4_predict.py	ML prediction charts
ml/	Module 5 — machine learning
predictor.py	LinearRegression: predict next month spend
anomaly.py	flag unusual spends vs historical average
logs/	auto-generated log files go here
exports/reports/	generated CSV and PDF reports saved here
tests/	unit tests for each module
config.py	DB path, log level, constants
requirements.txt	all pip packages

.env

secret keys — NEVER commit to GitHub

README.md

project description for GitHub

Target audience

This product solves a universal problem: people don't know where their money goes. The Indian market has ~180M young adults aged 18–35, fewer than 10% of whom use any budgeting tool. That is a massive untapped opportunity.

College students & freshers (18–24)

First time managing their own money. Hostel expenses, food, subscriptions — no idea where it goes. Will use the free tier and spread the word.

[High pain point] [Tech-savvy] [Free tier users] [Word-of-mouth]

Young working professionals (22–32)

Earning a first salary but lifestyle inflation hits fast. Want to save for a goal but can't track spending. Most likely to pay █99–199/month for the Pro tier.

[Will pay █99–199/mo] [Goal-oriented] [Salary earners] [Primary revenue]

Homemakers & household budget managers

Managing groceries, school fees, utilities, and EMIs in a notebook or WhatsApp. A clean dashboard would genuinely transform how they manage the household.

[Household expenses] [Category tracking] [Monthly budgets]

Freelancers & small business owners

Irregular income, multiple expense heads, need to track for taxes. Adding an income vs expense view makes this segment highly attractive and willing to pay more.

[Irregular income] [Tax tracking] [Will pay █299+/mo]

Monetization tiers

Free Tier	Pro █99/mo	Business █299/mo	One-time Sale
Basic tracking 3 months history Category charts	Spending predictions CSV export Full history Budget alerts	Multi-user support GST tagging Monthly PDF reports Priority support	Sell on Gumroad as a Python tool kit or GitHub sponsor

Build checklist — progress tracker

Use this checklist to track your progress. When you come back to this document and share what you have completed, your mentor can guide you from exactly where you left off.

#	Task	Module	Skills used	Done?
1	Create config.py with DB path, log level constants	M1	file handling	☐
2	Write schema.sql with 4 tables	M1	sqlite3	☐
3	Write models.py with 4 dataclasses	M1	OOP, dataclasses	☐
4	Write exceptions.py with custom exception classes	M1	exception handling	☐
5	Write db_manager.py with DatabaseManager class	M1	sqlite3, logging, OOP	☐
6	Set up logging in config.py	M1	logging	☐
7	Write seed_data.py and run it — verify rows in DB browser	M1	sqlite3, file handling	☐
8	Write expense_engine.py with generator functions	M2	generators, iterators	☐
9	Write decorators.py with @validate, @log, @require_auth	M2	decorators	☐
10	Write file_handler.py — CSV import and export	M2	file handling, exception handling	☐
11	Write reports.py using pandas for monthly aggregation	M3	pandas	☐
12	Write stats.py using numpy for averages and trend	M3	numpy	☐
13	Write charts.py — bar chart, pie chart, heatmap	M3	matplotlib, seaborn	☐
14	Write Flask app.py and routes.py with REST endpoints	M4	flask	☐
15	Write background.py for async report generation	M4	multithreading	☐
16	Build Streamlit overview page with metric cards	M4	streamlit	☐
17	Build Streamlit expenses page (add/edit/delete)	M4	streamlit, flask	☐
18	Build Streamlit charts page with all visualizations	M4	streamlit, seaborn	☐
19	Write predictor.py with LinearRegression model	M5	scikit-learn	☐
20	Write anomaly.py to flag unusual expenses	M5	scikit-learn, numpy	☐
21	Build Streamlit predictions page	M5	streamlit	☐
22	Write unit tests for all modules	ALL	pytest	☐
23	Write README.md and push to GitHub	ALL	git	☐

Skills you already have — mapped to the project

Your Skill	Where it is used in this project
Python OOP	models.py (User, Expense classes), DatabaseManager class
File Handling	file_handler.py (CSV import/export), schema.sql reader
Exception Handling	exceptions.py, all try/except in db_manager.py and routes.py
Generators & Iterators	expense_engine.py, get_expenses() yields rows one by one
Decorators	decorators.py: @validate_input, @log_activity, @require_auth
NumPy	stats.py: averages, standard deviations, trend arrays
Pandas	reports.py: groupby month, pivot tables, DataFrames from DB
Matplotlib	charts.py: bar charts, trend lines, monthly comparisons
Seaborn	charts.py: heatmaps (day vs category), distribution plots
SQLite3	db_manager.py: entire database layer
Logging	config.py setup, then used in every module
Memory Management	generators instead of loading all rows into RAM at once
Multithreading	background.py: generate PDF reports without blocking UI
Flask (soon)	api/app.py and routes.py: REST backend
Streamlit (soon)	dashboard/: all 4 frontend pages

Additional skills to learn (minimal effort, big impact)

Skill	Why you need it	Time to learn	Resource
scikit-learn basics	Linear regression for spending prediction	2–3 days	Official docs + YouTube
REST API design	Structure your Flask routes properly	1 day	Flask docs
bcrypt	Hash passwords securely	2 hours	pip install bcrypt
pytest basics	Write unit tests for each module	1 day	pytest.org

How to use this document

This blueprint is your single source of truth for the entire project. Here is exactly how to use it:

Step 1	Read pages 1–3 fully before writing any code. Understand what you are building and why.
Step 2	Create the folder structure from page 4. Just the empty folders and blank files.
Step 3	Start with Module 1 (database). Follow the checklist on page 6, items 1–7.
Step 4	After each module, tick off the checklist items. Commit to GitHub after each module.
Step 5	When stuck, share the relevant code + error with your mentor and mention which checklist item you are on.
Step 6	When you return to your mentor, say: 'I have completed checklist items 1 to X' — this is enough context to continue.
Step 7	After Module 4, deploy to a free host (Streamlit Cloud or Render.com) and share the link.
Step 8	After Module 5, write a LinkedIn post: 'I built a finance dashboard with ML predictions using Python.' It will get attention.

Remember: the goal is a product, not just a project.

Every line of code you write should serve a real user. Build it like someone is actually going to use it — because once you put it on GitHub and LinkedIn, they will.